

EA & MDA

Von Diagrammen zu Enterprise-Fähigkeit –
Standards, Capabilities und die Trennung von Business und Implementierung.

Lehrskript – Tag 5 von 16

Lernpfad:
Wissen → Anwendung → Management

Dozent:
Can Siebert

Bearbeitungsdauer:
1 Kurstag

Format:
Theorie · Fallstudie · Coaching

Lernziele dieses Tages

Gestern haben wir das Repository als Betriebssystem für Prozessmodelle aufgesetzt. Heute heben wir die Perspektive an: weg vom einzelnen Diagramm, hin zur *unternehmensweiten Modellierungsfähigkeit*. Drei Themen führen das zusammen – kollaborative Tools mit Standards (Visio/Lucidchart als Stellvertreter), Enterprise Architecture als Steuerungsrahmen und Model-Driven Architecture als Übersetzungslogik zwischen Business-Entscheidung und Implementierungsdetail.

L1 – Wissen (Reproduktion und Einordnung)

- Den Unterschied zwischen Einzeldatei-Modellierung und kollaborativer Plattform-Modellierung benennen (Versionen, Kommentare, Review-Workflows, Templates, Shapesets).
- Enterprise Architecture (EA) toolneutral einordnen: Capabilities, Prozesse, Applikationen, Daten, Technologie.
- TOGAF-ADM und Zachman-Framework als orientierende Rahmenwerke einordnen, ohne sie auswendig zu können.
- MDA-Ebenen (CIM, PIM, PSM) sauber abgrenzen und ihre Verantwortungsbereiche benennen.

L2 – Anwendung (Transfer auf konkrete Situationen)

- Eine **Modelling Charter** mit fünf nicht verhandelbaren Standards und drei Freiheitsgraden formulieren.
- Eine Capability Map mit 8 bis 12 Capabilities für ein konkretes Geschäftsfeld erstellen und priorisieren.
- Den MDA-Pfad für eine priorisierte Capability skizzieren: CIM (Business-Ziel + Scope), PIM (Service-/Datenlogik), PSM (Umsetzungsidee).
- Vier KPIs (zwei Outcome/Speed, zwei Risk/Compliance) mit Ownern festlegen.

L3 – Management (Entscheidung und Verantwortung)

- Modellierungs-Governance als *Produkt* verstehen – mit Product Owner, Release-Mechanik und Akzeptanzstrategie.
- EA als Priorisierungs- und Investitionsinstrument einsetzen, nicht als Dokumentations-Selbstzweck.
- Die Verantwortungsgrenze zwischen CIM (Business-Entscheidung) und PIM/PSM (Implementierungsdetail) bewusst ziehen.
- Toolstrategie und Governance-Strenges unter Unsicherheit entscheiden, mit Frühwarnsignalen zur Nachsteuerung.

Roter Faden

Ein Modellierungs-Werkzeug ist keine Strategie. Erst durch Standards, Capabilities und eine bewusste MDA-Grenze wird aus Diagrammen ein *Steuerungssystem*. Senior-Disziplin heißt: *Tool als Enabler, nicht als Methode – und Governance als Produkt, nicht als Dokument*.

1. Einstieg: Von Einzeldateien zu Enterprise-Fähigkeit

Es ist ein vertrautes Bild: Sechs Teams modellieren Prozesse. Jedes Team hat sein eigenes Tool, sein eigenes Vorgehen, seine eigenen Konventionen. Das Management klagt über mangelnde Vergleichbarkeit. Audit verlangt Versionsstände, die niemand zentral kennt. Die operativen Teams wiederum verlangen Flexibilität, weil sie die Eigenheiten ihrer Standorte und Produkte respektieren wollen. (Lankhorst, 2017; Allweyer, 2020)

Dieses Muster lässt sich nicht durch ein besseres Tool lösen. Es ist ein *Reifegrad-Problem*: Die Organisation hat Modellierung gelernt, aber noch keine **Enterprise-Modellierungsfähigkeit** aufgebaut. Heute schauen wir uns drei Bausteine an, die diese Fähigkeit ausmachen.

1.1 Drei Symptome fehlender Enterprise-Fähigkeit

1. **Tool-Wildwuchs.** Visio in Bereich A, Lucidchart in Bereich B, draw.io in Bereich C – jedes Modell ein eigener Datenfriedhof.
2. **Diskussionen drehen sich um Notation, nicht um Inhalt.** Statt über den Prozess wird über das Symbol gestritten.
3. **Business und IT reden aneinander vorbei.** Das Business spricht über “den Prozess”, die IT über “das System” – niemand übersetzt zwischen den Ebenen.

1.2 Drei Hebel für Enterprise-Fähigkeit

- **Kollaboration mit Standards.** Visio, Lucidchart, Signavio, ARIS und andere unterstützen heute gleichzeitiges Arbeiten, Kommentare, Versions-Workflows. Ohne klare Standards bleibt das aber “Zeichenprogramm mit Cloud”.
- **Enterprise Architecture.** Ein Rahmen, der Modelle in einen größeren Zusammenhang stellt: Welche Fähigkeiten haben wir, welche Prozesse leben sie, welche Applikationen unterstützen sie, welche Daten fließen?
- **Model-Driven Architecture.** Eine Übersetzungslogik, die Business-Entscheidungen sauber von Implementierungsdetails trennt – damit Technik später nicht “am Business vorbei” baut.

Eselsbrücke: S-C-E

Drei Bausteine, die aus Einzelmodellierung Enterprise-Fähigkeit machen:

Standards – Naming, Pflichtfelder, Freigaben.

Capabilities – was kann die Organisation, in welcher Tiefe?

Ebenen-Trennung – Business-Entscheidung vs. technische Umsetzung.

Wer S-C-E zusammen denkt, baut Modellierung als unternehmensweite Kompetenz auf – nicht als Tool-Skill.

2. Kollaboration mit Standards: Tools allein reichen nicht

Moderne Modellierungs-Tools (Visio, Lucidchart, Signavio, ARIS, Camunda Modeler, draw.io u. a.) haben sich in den letzten zehn Jahren stark weiterentwickelt. Was früher Einzeldateien waren, sind heute kollaborative Räume mit Echtzeit-Editing, Kommentaren, Versionierung und Review-Workflows. Diese Fähigkeiten machen aus einem Diagramm-Editor erst eine Modellierungs-*Plattform*. (Allweyer, 2020)

2.1 Was kollaborative Modellierung leistet

- **Gleichzeitiges Arbeiten.** Zwei Modeler an einem Modell, mit Konfliktauflösung.
- **Kommentare und Diskussionen** direkt am Diagramm-Element, statt in einer separaten E-Mail-Schleife.
- **Versionsstände** mit Vergleichsfunktion: was wurde wann, von wem geändert?
- **Review-Workflows.** Modell wird in Review gestellt, ein zweiter Modeler/Reviewer prüft, dokumentiert Feedback, gibt frei.
- **Templates und Shapesets.** Vordefinierte Strukturen und Symbol-Sets, die Konsistenz erleichtern.
- **Verlinkung** auf Artefakte (Policies, Arbeitsanweisungen, KPIs) – Modell als Hub.

2.2 Was Tools *nicht* leisten

Drei Dinge, die kein Tool kompensiert, egal wie modern es ist:

- Inkonsistente Benennungen. Wenn “Bestellung prüfen” neben “Prüfung Bestellung” steht, hilft kein Diff.
- Fehlende Owner. Ein Modell ohne benannten Verantwortlichen wird vom Tool freundlich verwaltet – und im Audit als Lücke aufgedeckt.
- Fehlende Definition of Done. Ohne klare Kriterien, wann ein Modell “fertig” ist, wandert es jahrelang im Status “Draft”.

2.3 Die Modelling Charter – eine Seite, fünf Standards, drei Freiheiten

Eine pragmatische **Modelling Charter** fasst die unternehmensweiten Spielregeln auf eine Seite zusammen. Sie hat zwei Teile: das, was nicht verhandelbar ist, und das, was lokal entschieden werden darf. (Fowler, 2003)

Fünf nicht verhandelbare Standards

1. **Naming-Regel.** Verb + Objekt für Aktivitäten und Funktionen; “X-to-Y” für E2E-Prozesse; Zustand (passiv) für Ereignisse.
2. **Pflicht-Metadaten.** Owner, Status, Version, Gültigkeit, letzte Änderung – in jedem Modell, ohne Ausnahme.
3. **Rollen sichtbar.** Lanes oder Organisationseinheiten in jedem ausführungsnahen Modell.
4. **Start und Ende eindeutig.** Jeder Pfad endet in einem benannten End-Ereignis; kein Pfad verliert sich.
5. **Änderungslog (Change Request).** Jede Änderung über einen strukturierten Antrag, mit Reviewer und Approver-Vermerk.

Drei Freiheitsgrade

- **Detailtiefe.** Innerhalb der Konvention darf jedes Team selbst entscheiden, wie tief modelliert wird – abhängig von Zweck und Zielgruppe.
- **Visual Style innerhalb des Templates.** Farb-Akzente, Layout, Lesefluss bleiben lokale Entscheidung, solange Pflichtelemente vorhanden sind.
- **Lokale Varianten.** Standorte oder Produktlinien dürfen Varianten als Attribut führen, ohne dass jedes Modell zentral übersetzt werden muss.

2.4 Freigabeprozess in vier Schritten

1. **Draft.** Modeler arbeitet, Annahmen explizit.
2. **Review.** Reviewer (mindestens vier Augen) prüft Konvention und Logik.
3. **Approved.** Owner gibt frei, mit Datum, Reviewer- und Approver-Name.
4. **Retired.** Bei Ablösung wird das Modell archiviert (nicht gelöscht).

Tool als Enabler, nicht als Strategie

Visio und Lucidchart sind Werkzeuge – so wie ein guter Stift kein guter Aufsatz ist. Strategie entsteht durch Standards, Freiheitsgrade und eine ehrliche Definition of Done. Ohne diese Elemente wird ein modernes Modellierungs-Tool zur “schnelleren Möglichkeit, Modelle zu produzieren, die niemand reviewen kann”.

2.5 Was sofort, was später – Entscheidungen unter Unsicherheit

In einer wachsenden Organisation lassen sich nicht alle Standards gleichzeitig einführen. Eine bewährte Reihenfolge:

- **Sofort (2 Wochen).** Naming-Regel + Pflicht-Metadaten. Wirkung hoch, Aufwand niedrig.
- **Kurzfristig (4–6 Wochen).** Review-/Approval-Workflow, Pilotbereich.
- **Mittelfristig (3 Monate).** Change Request mit Reviewer-Rollen, Variantenpolitik.
- **Längerfristig (6–12 Monate).** KPI-Verlinkung, Risikoklasse, Capability-Mapping.

3. Enterprise Architecture: Capabilities, Prozesse, Applikationen

Enterprise Architecture (EA) ist die Disziplin, die Geschäftsfähigkeiten, Prozesse, Informationen, Applikationen und Technologie konsistent zueinander in Beziehung setzt. Sie ist tool-neutral; jedes große Framework (TOGAF, Zachman, ArchiMate) ist im Kern ein Strukturierungsangebot. (The Open Group, 2022; Zachman, 1987; Lankhorst, 2017)

3.1 Die zentrale EA-Frage

EA beantwortet im Kern drei Fragen, die alle anderen einrahmen:

1. **Was können wir?** – Capabilities (Fähigkeiten der Organisation).
2. **Wie arbeiten wir?** – Prozesse.
3. **Womit?** – Applikationen, Datenobjekte, Technologie.

3.2 Typische EA-Sichten

- **Capability Map.** Eine hierarchische Sicht aller Fähigkeiten, typischerweise zwei bis drei Ebenen. Stabil über Reorganisationen hinweg.
- **Prozesslandkarte.** Die E2E-Prozesse, die Capabilities aktivieren. Kann sich häufiger ändern als die Capability-Sicht.
- **Applikationslandschaft.** Welche Systeme unterstützen welche Prozesse, welche Capabilities, welche Datenobjekte.
- **Datenobjekte.** Die Kerninformationsobjekte (Kunde, Vertrag, Auftrag, Konto) – inklusive Verantwortlichkeit (Data Owner).
- **Schnittstellen.** Wo fließen Daten zwischen Systemen, wo gibt es Brüche?

3.3 Capability Map – das Rückgrat

Eine Capability ist eine *Fähigkeit*, nicht ein Prozess und nicht eine Applikation. “Kunde identifizieren”, “Vertrag prüfen”, “Zugang provisionieren” sind Capabilities. “Onboarding durchführen” ist ein Prozess, “IAM-System” ist eine Applikation. Diese Unterscheidung ist nicht akademisch – sie macht den Unterschied zwischen einer stabilen Strategie-Sicht und einem flüchtigen Tool-Inventar. (Ross et al., 2006)

Beispiel “Digital Customer Onboarding”

Eine realistische Capability-Liste:

1. Marketing-Lead generieren.
2. Interessent qualifizieren.
3. Kunde identifizieren (KYC).
4. Vertrag prüfen und kalkulieren.
5. Bonität bewerten.
6. Vertrag abschließen.
7. Zugang/Account provisionieren.
8. Daten initial befüllen.
9. Training/Enablement durchführen.
10. Go-Live-Übergabe an Operations.

Top-3 mit höchstem Hebel

In diesem Beispiel sind die Capabilities mit höchstem Hebel auf Durchlaufzeit und Compliance:

- **Kunde identifizieren (KYC).** Gleichzeitig Compliance-kritisch und Hauptengpass im DLZ.
- **Bonität bewerten.** Externe Schnittstelle, häufige Fehlerquelle.
- **Zugang/Account provisionieren.** Technische Brücke zwischen Vertrieb und IT, oft mehrere Tage statt Stunden.

3.4 Datenobjekte und Risiken

Datenobjekt	Beschreibung	Typische Risiken
Kunde	Identifizierende und kontaktbezogene Informationen	Datenqualität, Datenschutz, Dubletten
Vertrag	Rechtlich verbindliche Vereinbarung	Versionierung, Schriftform, Archivierung
Identitätsnachweis	Dokument zur KYC-Erfüllung	Echtheit, Fristen, Audit-Trail
Account	Technische Zugangsidentität	Rollenmodell, Provisioning, Off-Boarding

Capability vs. Prozess vs. Applikation

Eine Capability ist eine *Fähigkeit*: “Wir können Kunden identifizieren.” Ein Prozess ist eine *Ablauffolge*, die diese Fähigkeit aktiviert: “Onboarding durchführen.” Eine Applikation ist ein *Werkzeug*, mit dem der Prozess die Capability operationalisiert: “IAM-System.” Capabilities sind stabil. Prozesse wandern. Applikationen kommen und gehen.

3.5 TOGAF-ADM und Zachman – zwei Hilfsstrukturen

TOGAF (The Open Group Architecture Framework) bringt mit dem **ADM** (Architecture Development Method) einen iterativen Vorgehensplan – von Architecture Vision über Business Architecture, Information Systems Architectures, Technology Architecture, Opportunities/Solutions, Migration Planning, Implementation Governance bis Architecture Change Management. ADM ist kein Wasserfall, sondern ein Zyklus.

Das **Zachman-Framework** ordnet Architekturen entlang sechs Fragen (Was, Wie, Wo, Wer, Wann, Warum) und sechs Perspektiven (Executive, Business Management, Architect, Engineer, Subcontractor, Functioning Enterprise) – eine 6×6-Matrix als Klassifikationsschema. (Zachman, 1987)

Praktischer Umgang: beide Frameworks lassen sich *leichtgewichtig* nutzen – als Strukturanker, nicht als Vorgabe. Ein praktischer Anteil von 20 % an Inhalten reicht, um Diskussionen zu strukturieren. Der Rest ist akademisch interessant, aber im Tagesgeschäft selten relevant.

4. Model-Driven Architecture (MDA)

MDA ist ein OMG-Standard, der eine zentrale Übersetzungslogik anbietet: vom Business-Anliegen zur technischen Umsetzung in drei sauber getrennten Ebenen. (OMG, 2014; Kleppe et al., 2003)

4.1 Die drei Ebenen

1. **CIM – Computation Independent Model.** Die Business-Sicht. Ziele, Prozesse, Begriffe, Scope, Datenobjekte aus Fachperspektive. Hier entscheidet das Business, was es will – ohne Rücksicht auf Plattformen.
2. **PIM – Platform Independent Model.** Die logische System- und Service-Sicht. Welche Services, welche Datenflüsse, welche Fehlerfälle, welche Schnittstellen – aber ohne konkrete Plattform.
3. **PSM – Platform Specific Model.** Die konkrete technische Umsetzung. Welche konkrete Cloud-Plattform, welche konkrete API-Technologie, welche konkrete Deployment-Variante.

4.2 Verantwortungsfähigkeit – wer entscheidet was

CIM Business / Process Owner / Product Owner entscheiden.

PIM Architekten / Lösungs-Engineers / Senior-Fachleute entwerfen, in Abstimmung mit Business.

PSM IT-Teams / Plattform-Engineers / Spezialisten setzen um – mit Architektur-Review.

4.3 Was im CIM stehen muss

Ein CIM, das mehr als eine Überschrift ist, enthält:

- Das Business-Ziel (messbar, falls möglich).
- Den Scope (was gehört dazu, was nicht).
- Die wichtigsten Datenobjekte mit Pflichtfeldern.
- Die zentralen Prozesse mit Hauptpfad.
- Die nicht verhandelbaren Regeln (Compliance, Security, Governance).
- Die Erfolgskriterien.

Beispiel “Digital Customer Onboarding” CIM

- **Business-Ziel:** “Kunde in < 24 h aktiv, ohne manuelle Re-Eingaben.”
- **Scope:** Komplettes Onboarding ab unterschriebenem Vertrag bis Go-Live; explizit *nicht* im Scope: Marketing-Lead-Generierung.
- **Datenobjekt “Kunde”:** Pflichtfelder Name, Adresse, Identifikationsnachweis, Vertragsreferenz.
- **Kernprozesse:** KYC, Bonität, Vertragsabschluss, Provisioning, Training.
- **Nicht verhandelbar:** KYC vor jedem Provisioning; Audit-Logging Pflicht; Datenlöschung gemäß DSGVO.
- **Erfolgskriterien:** DLZ-Median < 24 h; Compliance-Findings = 0; Kunden-NPS > +30.

4.4 Was im PIM stehen muss

- Die logischen *Services*, die das CIM operationalisieren (“VerifyIdentity”, “CheckCreditScore”, “CreateAccount”, “AssignRoles”).
- Die *Datenflüsse* zwischen Services – welche Felder gehen wohin.
- Die *Fehlerfälle* und deren Behandlung (fehlende Dokumente, Service nicht erreichbar, Negative-Ergebnisse).
- Die *Service-Verträge* (Input, Output, Idempotenz, Versionierung – noch nicht plattformspezifisch).

4.5 Was im PSM stehen muss

- Die konkrete *Plattformwahl* (Cloud-Provider, Region, Tenant).
- Die konkreten *Schnittstellen* (REST/gRPC/AsyncAPI, Auth-Verfahren, Endpoints).
- Die *Deployment-Strategie* (Container, IaC, CI/CD).
- *Audit-Logging* mit konkretem Logging-Stack.
- Die *Sicherheits-Konfiguration* (Secrets, Netzwerk-Policies).

Eselsbrücke: C-P-S

Die drei MDA-Ebenen, von Business nach Technik:

CIM – Was wollen wir? (Business)

PIM – Wie logisch? (Service-/Datensicht)

PSM – Womit konkret? (Plattform-Umsetzung)

Wer CIM und PSM vermischt, bekommt entweder ein Architekturpapier, das Business nicht versteht, oder ein Business-Dokument, das die IT nicht bauen kann.

4.6 Die Grenze als Senior-Disziplin

Die häufigste MDA-Falle ist *nicht*, dass die Ebenen fehlen – sondern dass sie unscharf vermischt werden. Drei typische Symptome:

- Das CIM enthält Plattform-Details (“wir nutzen AWS Step Functions”). Folge: Business bindet sich an Technik, ohne zu wissen, was sie damit ausschließt.
- Das PIM fehlt komplett – Business springt direkt auf PSM. Folge: Lock-in, kein zweiter Anbieter denkbar.
- Das PSM widerspricht dem CIM. Folge: gebaut wird etwas, das das Business nicht wollte – aber jeder dachte, der andere hätte zugestimmt.

MDA als Verantwortungsgrenze

MDA ist nicht primär eine Notations-Vorgabe, sondern eine *Verantwortungsgrenze*: Was darf das Business entscheiden, was die Architektur, was die Implementation? Diese Trennung schützt das Business vor verfrühter Technik-Bindung – und die IT vor unklaren Business-Anforderungen.

5. Fallstudie: RetailGroup Europe – Charter, Capabilities und MDA-Pfad

Wir betrachten ein realistisches Szenario: *RetailGroup Europe* hat Filialgeschäft und Online-Vertrieb in mehreren Ländern. Es gibt viele Prozessdokumente, aber keine Vergleichbarkeit. Digitalisierungsinitiativen stocken, weil Business und IT aneinander vorbeireden.

5.1 Ausgangslage und Restriktionen

- Zeit: 10 Wochen bis Audit-Vorbereitung.
- Budget: 100 000 €.
- Zwei parallele IT-Projekte konkurrieren um Kapazität.
- Verantwortlichkeiten zwischen Zentrale und Länderorganisationen unklar.

5.2 Schritt 1 – Modelling Charter (Standards + Freiheiten + Freigabe)

Fünf Standards (nicht verhandelbar)

1. Naming-Regel: Verb + Objekt für Aktivitäten, “X-to-Y” für E2E.
2. Pflicht-Metadaten: Owner, Status, Version, Standort, Risikoklasse.
3. Rollen sichtbar (Lanes/Organisationseinheiten in jedem ausführungsnahen Modell).
4. Start- und End-Ereignis verpflichtend.
5. Änderungslog über Change Request.

Drei Freiheitsgrade

- Detailtiefe nach Bedarf und Zweck.
- Visual Style innerhalb des Templates.
- Lokale Varianten als Attribute.

Freigabe-/Änderungsprozess

Draft → Review (Modeler + Reviewer, vier Augen) → Approved (Owner) → Retired (nach Ablösung).

5.3 Schritt 2 – Capability Map (Auszug, 12 Capabilities)

1. Sortiment planen.
2. Lieferanten managen.
3. Beschaffen.
4. Lager bewirtschaften.
5. Filialprozesse steuern.
6. Kunden gewinnen.
7. Online-Verkauf abwickeln.

8. Bezahlen / Kassieren.
9. Identifizieren / Authentifizieren.
10. Liefern (eigene + Partner).
11. Reklamieren und Retournieren.
12. Reporten und Auditieren.

5.4 Schritt 3 – Zwei Digitalisierungsprioritäten

- **Identifizieren/Authentifizieren** (Capability 9). Begründung: Hebel auf DLZ Onboarding, Compliance-relevant (Geldwäsche, Altersprüfung), Wiederverwendung in mehreren Prozessen.
- **Zugang/Account Provisioning** (Capability nahe 7/9). Begründung: Hebel auf Customer Lifetime Value, DLZ-Killer für Neukunden, Schnittstelle zu IT-Architektur als Folgeeffekt.

5.5 Schritt 4 – MDA-Pfad für “Identifizieren”

CIM

- **Ziel:** “Kundenidentität in < 5 min verifizierbar, in 95 % der Fälle digital.”
- **Scope:** Online und Filiale (gleicher Anker für beide Kanäle).
- **Datenobjekt “Kunde”:** Pflichtfelder Vor-/Nachname, Geburtsdatum, Adresse, eindeutige ID.
- **Regeln:** KYC-Pflicht über bestimmten Transaktionswert; DSGVO-konforme Speicherung; Audit-Logging.

PIM

- Service “VerifyIdentity” (Input: Kunde-Stammdaten + Identitätsnachweis; Output: Verifikations-Status + Verifikations-ID + Zeitstempel).
- Datenfluss: Kunde-Stammdaten kommen aus CRM, Identitätsnachweis aus Filial-Scanner oder Online-Upload.
- Fehlerfälle: schlechte Bildqualität (Rückfrage), Unstimmigkeit Daten/Dokument (manuelle Prüfung), Verifikations-Service nicht erreichbar (Fallback auf manuelle Bearbeitung).

PSM

- Konkrete Plattform: bestehendes IAM-System plus Verifikations-Service eines Anbieters.
- REST-API mit OAuth-2.0, Versionierung in URL.
- Audit-Logging zentral, mit unveränderlichem Speicher.
- Deployment in Container, IaC für Test-/Produktivumgebung.

5.6 Schritt 5 – Vier KPIs (zwei Outcome, zwei Risk/Compliance)

- **Lead Time Onboarding** (Outcome). Ziel: Median < 24 h.
- **First-Pass-Yield** (Outcome). Anteil Fälle, die ohne Rückfrage durchlaufen. Ziel: > 85 %.
- **Compliance-Findings** (Risk). Audit-Befunde pro Quartal. Ziel: 0.
- **Rework-Rate** (Risk). Anteil Fälle mit manueller Nachbearbeitung. Ziel: < 10 %.

5.7 Lessons aus der Fallstudie

- Standards entstehen nicht in einem Workshop – sie entstehen durch *wiederholte* Anwendung und Konsequenz.

- Capabilities sind robuster als Prozesse: eine gute Capability Map überlebt zwei Reorganisationen.
- MDA wirkt erst dann als Steuerungslogik, wenn die Grenze *durchgehalten* wird – auch unter Zeitdruck.

6. Management-Perspektive: Modellierung als Produkt

Auf Wissens-Ebene sind Standards und Frameworks Lernstoff. Auf Anwendungs-Ebene sind sie Handwerk. Auf Management-Ebene wird Modellierung selbst zum *Produkt* – mit einem Owner, Releases, Akzeptanzstrategie und messbaren Erfolgskriterien.

6.1 Zwei zentrale Zielkonflikte

1. **Tool-Streng** (ein Standardtool) vs. **Föderation** (mehrere zugelassene Tools). Ein zentrales Tool maximiert Vergleichbarkeit – und Widerstand. Eine Föderation schont Akzeptanz – und multipliziert Pflege. Praktische Lösung: *ein* primäres Tool + ein bis zwei zugelassene Sekundär-Tools mit Export-Pflicht in gemeinsamen Standard.
2. **Governance-Streng** vs. **Schnelligkeit**. Strenge Reviews machen Modelle audit-fest – und langsam. Lockere Reviews machen Modelle schnell – und unzuverlässig. Empfehlung: harte Regeln für die *vier nicht verhandelbaren Standards*, lokale Freiheit innerhalb dieser Leitplanken. (Ross et al., 2006)

6.2 Modellierung als Produkt – Product-Owner-Modell

- Ein benannter **Product Owner** des Modellierungs-Systems (häufig: Head of Enterprise Architecture oder Chief Process Officer).
- Ein **Release-Rhythmus** der Standards: zwei Mal pro Jahr Updates, mit Vorlauf und Schulungs-Snippets.
- Eine **Review-Mechanik**: Reviewer-Pool, Vier-Augen-Pflicht, regelmäßiges Reviewer-Treffen.
- Ein **Feedback-Kanal** aus Fachbereichen: was funktioniert, was nicht, welche Standards wirken aufwendig ohne Nutzen?

6.3 EA als Priorisierungs- und Investitionsinstrument

EA-Capabilities sind nicht primär Dokumentation – sie sind **Priorisierungssprache**. Wenn ein Investment-Antrag auf zwei Capabilities einzahlt, die in der Capability Map als kritisch markiert sind, wird er anders bewertet als ein Antrag auf eine Capability, die strategisch keine Priorität hat. (Ross et al., 2006)

Drei Anwendungsfelder:

- **Investitionspriorisierung**. Welche Capabilities erhalten Budget?
- **Standardisierungsentscheidungen**. Wo lohnt sich ein Standard, wo darf lokal divergiert werden?
- **Risiko-/Compliance-Steuerung**. Welche Capabilities sind compliance-kritisch und müssen entsprechend gemonitort werden?

6.4 MDA-Grenze unter Zeitdruck

Die häufigste Erosion der MDA-Grenze passiert unter Zeitdruck. Ein Vorstand fragt nach einer Lösung, und in der Hektik werden CIM und PSM vermischt. Das ist nicht Bössartigkeit, sondern operative Notwendigkeit – und genau hier braucht es Senior-Disziplin:

- Wenn ein CIM unterschrieben werden muss, ohne dass Plattformfragen geklärt sind: explizit als “offen” markieren, nicht durch spontane Entscheidungen schließen.
- Wenn das PSM ohne PIM gebaut wird: dokumentieren, dass dies eine bewusste MVP-Entscheidung ist, mit Trigger für ein nachträgliches PIM-Reverse-Engineering.

6.5 Entscheidungen unter Unsicherheit

- **Tool-Strategie.** Standardtool jetzt, später öffnen? Oder fördert starten, später konsolidieren? Beide Wege haben sechsstelligen Folgekosten. Schriftliche Entscheidung, verworfene Alternative dokumentieren, Frühwarnsignal definieren (z. B. “mehr als drei aktive Tools → Konsolidierungs-Workshop”).
- **Governance-Strengung.** Minimal-Set jetzt, ergänzen nach Akzeptanz? Oder vollständig jetzt mit Schmerz? Empfehlung: Minimum-Set zuerst, mit Trigger zur Verschärfung bei wiederholten Audit-Findings.

Senior-Shift – vom Diagramm zur Architektur-Verantwortung

Der Wechsel von “ich modelliere einen Prozess” zu “ich verantworte ein Modellierungssystem” ist erkennbar an drei Symptomen: man entscheidet die Standards vor der ersten Datei; man kann die MDA-Grenze auch unter Druck verteidigen; man sieht Capabilities als langlebige Sprache, nicht als Reporting-Element.

7. Take-aways aus Tag 5

1. **Tool ist Enabler, nicht Strategie.** Visio/Lucidchart erlauben Kollaboration – aber Standards machen daraus Steuerung.
2. **Modelling Charter auf einer Seite.** Fünf nicht verhandelbare Standards, drei Freiheitsgrade, Freigabe-/Änderungsprozess – nicht mehr, nicht weniger.
3. **EA = Capabilities + Prozesse + Applikationen + Daten.** Capabilities sind das stabile Rückgrat – robuster als Prozesse, langlebiger als Applikationen.
4. **TOGAF und Zachman leichtgewichtig nutzen.** Strukturanker, keine Wasserfall-Vorgabe – 20 % des Frameworks reichen meist.
5. **MDA = CIM (Business) + PIM (logisch) + PSM (konkret).** Drei Ebenen, drei Verantwortungen, eine durchgehaltene Grenze.
6. **Capability-Priorisierung als Steuerungssprache.** Investitionen, Standards und Risiken folgen der Capability Map.
7. **Governance als Produkt.** Ein Owner, ein Rhythmus, ein Feedback-Kanal – sonst altert das System.
8. **Entscheidungen unter Unsicherheit schriftlich.** Verworfen Alternative + Frühwarnsignal + Trigger zur Re-Evaluation.

8. Literatur

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

Allweyer, T. (2020).

BPMN 2.0 – Business Process Model and Notation: Einführung in den Standard für die Geschäftsprozessmodellierung (4. Aufl.). BoD.

Fowler, M. (2003).

UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley.

Kleppe, A., Warmer, J., & Bast, W. (2003).

MDA explained: The model driven architecture – practice and promise. Addison-Wesley.

Lankhorst, M. (2017).

Enterprise architecture at work: Modelling, communication and analysis (4th ed.). Springer.
<https://doi.org/10.1007/978-3-662-53933-0>

Object Management Group. (2014).

Model Driven Architecture (MDA) Guide rev. 2.0 (ormsc/2014-06-01). <https://www.omg.org/cgi-bin/doc?ormsc/14-06-01>

Ross, J. W., Weill, P., & Robertson, D. C. (2006).

Enterprise architecture as strategy: Creating a foundation for business execution. Harvard Business School Press.

The Open Group. (2022).

TOGAF® standard, version 9.2: A standard of The Open Group. The Open Group. <https://www.opengroup.org>

Zachman, J. A. (1987).

A framework for information systems architecture. *IBM Systems Journal*, 26(3), 276–292.
<https://doi.org/10.1147/sj.263.0276>

9. Glossar

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

Applikationslandschaft

Sicht auf Systeme einer Organisation und ihre Zuordnung zu Capabilities, Prozessen und Datenobjekten.

ArchiMate

Open-Group-Notationssprache für Enterprise-Architektur-Modelle.

Capability

Fähigkeit der Organisation (“Kunde identifizieren”). Stabil über Reorganisationen.

Capability Map

Hierarchische Sicht aller Capabilities einer Organisation, meist zwei bis drei Ebenen.

Change Request

Strukturierter Antrag zur Änderung eines Modells, mit Reviewer und Approver.

CIM

Computation Independent Model. Business-Sicht eines MDA-Pfads – Ziele, Prozesse, Begriffe, Regeln.

Datenobjekt

Kerninformationsobjekt (Kunde, Vertrag, Konto) inklusive Data Owner.

Definition of Done

Liste prüfbarer Kriterien, ab denen ein Modell als fertig gilt.

EA

Enterprise Architecture. Disziplin der konsistenten Beziehung von Capabilities, Prozessen, Applikationen, Daten, Technologie.

Freiheitsgrad

Bereich der Modellierungs-Entscheidung, der bewusst lokal bleibt.

Lucidchart

Cloud-basiertes kollaboratives Modellierungs-Tool, breit für Prozess- und EA-Diagramme verwendet.

MDA

Model Driven Architecture. OMG-Standard zur Trennung von Business (CIM), logischer Architektur (PIM) und konkreter Umsetzung (PSM).

Modelling Charter

Einseitiges Dokument mit den unternehmensweiten Standards und Freiheitsgraden für Modellierung.

PIM

Platform Independent Model. Logische Service- und Datensicht in MDA, unabhängig von Plattform.

PSM

Platform Specific Model. Konkrete technische Umsetzung in MDA.

Product Owner (Modellierungs-System)

Verantwortliche Person für Standards, Releases und Akzeptanz des Modellierungs-Systems.

Prozesslandkarte

E2E-Sicht auf alle Kernprozesse einer Organisation; Mappt Capabilities auf Abläufe.

Reviewer

Rolle, die ein Modell vor Approval auf Konventionen und Logik prüft.

Shapeset

Vordefinierte Symbolbibliothek, die in einem Modellierungs-Tool Konsistenz erleichtert.

Standard (nicht verhandelbar)

Regel in der Modelling Charter, die unternehmensweit ohne Ausnahme gilt.

TOGAF

The Open Group Architecture Framework. Verbreitetes EA-Rahmenwerk mit der Architecture Development Method (ADM).

Visio

Microsoft-Modellierungs-Tool, weit verbreitet in Unternehmen für BPMN-, EPC- und EA-Diagramme.

Zachman-Framework

Klassifikationsschema (6×6-Matrix) für EA-Artefakte; ursprünglich Zachman, 1987.